# 🧠 LangGraph Quiz

## 📌 Section A: MCQs (1 mark each)

**1. What is LangGraph primarily used for?**
a) Data visualization
b) Building graph-based AI workflows
c) Database management
d) Web scraping
**Answer: b) Building graph-based AI workflows**

---

**2. In LangGraph, nodes represent:**
a) Database tables
b) Functions or steps in a workflow
c) UI components
d) API keys
**Answer: b) Building graph-based AI workflows**

---

**3. What is the main advantage of graph-based workflows?**
a) Linear execution only
b) No need for state
c) Flexible branching and looping
d) Faster internet speed
**Answer: b) Functions or steps in a workflow**

---

**4. What does "state" represent in LangGraph?**
a) Server location
b) Memory shared across nodes
c) Programming language
d) UI design
**Answer: c) Flexible branching and looping**

**5. Which component decides the next step in a LangGraph workflow?**
a) Tokenizer
b) Edge
c) Prompt
d) Dataset
**Answer: b) Edge**

---

**6. A multi-step agent is best described as:**
a) A single API call
b) A system executing multiple connected steps
c) A database query
d) A static webpage
**Answer: b) A system executing multiple connected steps**

---

**7. What is the purpose of conditional edges in LangGraph?**
a) To delete nodes
b) To control flow based on conditions
c) To store data
d) To train models
**Answer: b) A system executing multiple connected steps**

---

**8. Which of the following is required to define a graph in LangGraph?**
a) Nodes and edges
b) Only APIs
c) Only datasets
d) Only UI components
**Answer: a) Nodes and edges**

---

**9. StateGraph in LangGraph is used for:**
a) Styling UI
b) Managing workflow state and transitions
c) Creating databases
d) Sending emails
**Answer: b) Managing workflow state and transitions**

---

**10. What is the output of a node in LangGraph?**
a) Only text
b) Updated state
c) Images only
d) HTML code
**Answer:b)Updated State**

---

## 📌 Section B: Short Answer (2–3 marks each)

**11. Explain the concept of graph-based workflows in LangGraph.**
**Answer:**

Graph-based workflows represent processes as a network of nodes and edges. Each node performs a specific task, and edges define the flow between them. This allows flexible execution, including branching and looping, unlike fixed linear workflows.

---

**12. What is state management and why is it important in multi-step agents?**
**Answer:**

Graph-based workflows represent processes as a network of nodes and edges. Each node performs a specific task, and edges define the flow between them. This allows flexible execution, including branching and looping, unlike fixed linear workflows.

---

**13. Differentiate between nodes and edges in LangGraph.**
**Answer:**

Nodes are individual functions or steps that perform tasks, while edges define how the flow moves from one node to another. Nodes do the work, and edges control the execution path.

---

**14. What is a StateGraph and how does it help in building agents?**
**Answer:**
StateGraph is a structure used to define workflows with nodes, edges, and shared state. It helps in building agents by managing transitions between steps and ensuring data flows correctly across the system.

---

**15. Give one real-world example where a multi-step agent can be useful.**
**Answer:**

A customer support chatbot that classifies user queries and routes them to billing, technical, or general support is a good example of a multi-step agent.

---

## 📌 Section C: Coding-Based Questions (4–5 marks each)

**16. Consider the following code:**

```
def step1(state):
    return {"msg": "Hello " + state["name"]}
```

👉 What is happening in this function? Explain.
**Answer:**
This function takes the input state, reads the value of "name", and creates a greeting message by combining "Hello " with the name. It then returns the updated state containing the new message.

---

**17. Write a simple LangGraph workflow with two steps:**

- Step 1: Take user input
- Step 2: Generate a response

(You can write pseudocode)
**Answer:**

State = {input, response}

Node1: get_input

- Take user input

- Store in state["input"]

Node2: generate_response

- Create reply using state["input"]

- Store in state["response"]

Flow:

get_input → generate_response → END

---

**18. How would you add a conditional step in a LangGraph workflow? Explain with logic.**
**Answer:**
A conditional step is added using a function that decides the next node based on the state.
Example: if "exam" in question:
go to exam_node
else:
go to assignment_node

In LangGraph, this is implemented using conditional edges, where the function returns the name of the next node based on conditions.

---

## 📌 Section D: Case Study (5–8 marks)

**19. Design a multi-step agent using LangGraph for a "Student Assistant Bot" that:**

- Takes a question
- Checks if it is about assignments or exams
- Routes to the correct handler
- Returns a response

👉 Explain:

- Nodes
- Edges

State
**Answer:**

**State:**

- question → Student's query
- category → "assignment" or "exam"
- response → Final reply

**Nodes:**

1. Input Node → Takes student question
2. Router Node → Classifies question
3. Assignment Node → Handles assignment queries
4. Exam Node → Handles exam queries

**Edges:**

- Input → Router
- Router → Assignment Node (if assignment)

- Router → Exam Node (if exam)
- Assignment/Exam → END

  **Working:**
  The student enters a question. The router checks whether it is related to assignments or exams and sends it to the appropriate node, which generates a response.

---

## 📌 Section E: Advanced Conceptual (5 marks)

**20. Compare traditional linear AI pipelines with graph-based agent workflows. Why is LangGraph better for complex systems?**
**Answer:**

| Linear Pipeline | Graph-Based Workflow |
|---|---|
| Fixed sequence of steps | Flexible structure |
| No branching or looping | Supports branching and loops |
| Less adaptable | Highly dynamic |
| Suitable for simple tasks | Suitable for complex systems |

**Why LangGraph is better:**
LangGraph allows dynamic decision-making, supports multiple paths, maintains state across steps, and enables complex multi-agent systems, making it ideal for real-world AI applications.